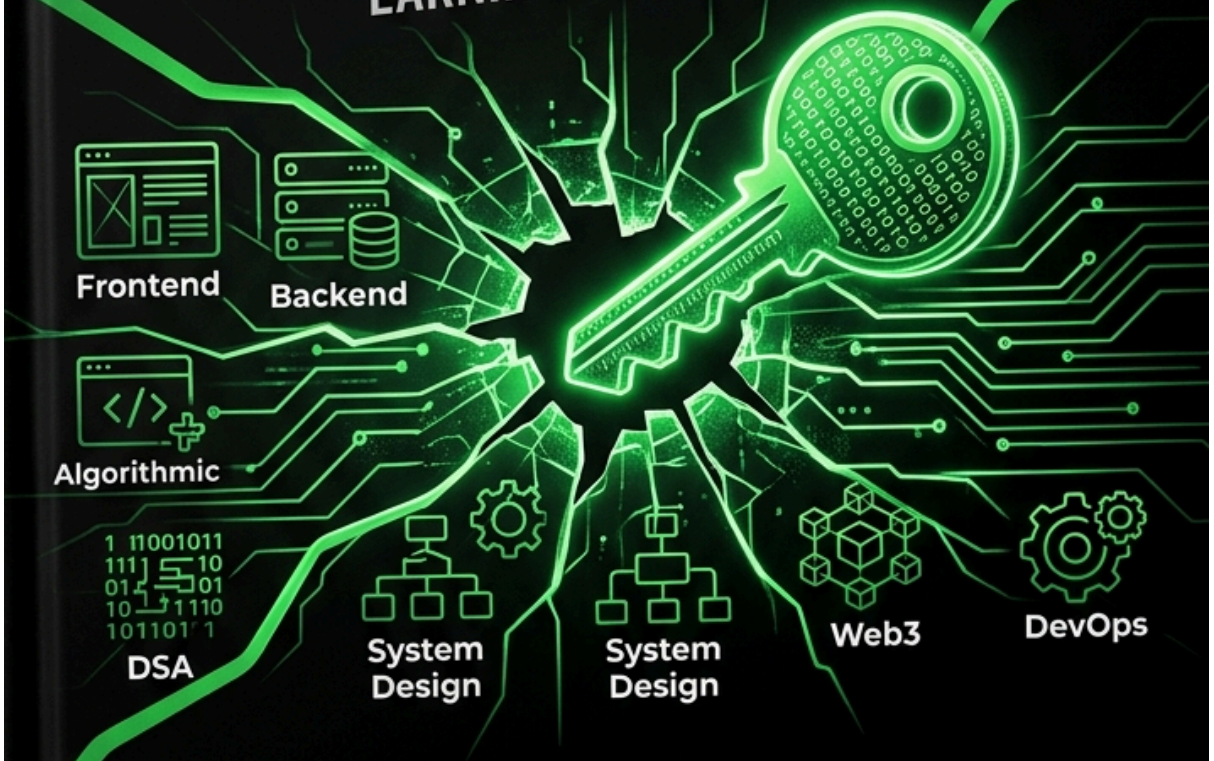
THE CRACKED DEV PLAYBOOK

A REAL-WORLD GUIDE TO BREAKING INTO TECH, CHOOSING YOUR PATH, AND EARNING SIX FIGURES



ABDULRAHMAN RAUFU

A KODED PUBLICATION FROM STACKD

The Cracked Dev Playbook

© Abdulrahman Raufu 2026

Forward

There's a guy I know named Ibraheem. Regular Nigerian student, no computer science background, no uncle in Silicon Valley, no connections, no prior experience. Just a phone, a laptop that had seen better days, and a quiet feeling that life was moving and he was standing still.

He reached out to us one evening and said something I've heard a hundred times since. "I want to learn tech but I don't even know where to start. And honestly I think it might be too hard for me."

We didn't give him a course link. We didn't send him a YouTube playlist. We sat with him, figured out which path made sense for his goals, gave him structure, and made him build real things from day one.

That same guy is now working at a startup. Earning six figures. He started from absolute zero.

I wrote this book because that conversation, the one that changed Ibraheem's direction, shouldn't be rare. Too many people are sitting exactly where he was, talented and hungry but with nobody in their corner to show them how this actually works.

This is that conversation. In a book. For you.

By the time you finish reading this, you'll know which track is right for you, what your first thirty days should look like, and exactly what to do next.

Let's get into it.

~ Koded from StackD

The Industry is Paying. Are You Ready?

Let me tell you something that doesn't get said enough. The tech industry is paying crazy money right now. Not in five years. Not when the economy gets better. Right now, today, as you're reading this.

Entry level developers working remotely are clearing ₦500,000 a month. Junior engineers at startups are earning more in a month than some of their parents earn in a year. Web3 developers, cloud engineers, backend developers, the demand is insane and the supply of people who actually know what they're doing is still embarrassingly low.

I'm not telling you this to hype you up. I'm telling you this because most people have no idea it's happening. They see tech as something for a specific type of person, someone who studied computer science, someone who has been coding since they were nine, someone who just gets it naturally. And because they believe that story, they never even try.

But here's what I've seen over and over again. The people eating in this industry are not always the smartest people in the room. They are the people who started. They picked a path, they showed up every day, they built things, and they got good. Full stop.

The gap between you and a developer earning ₦500,000 a month is not talent or big tech certificates. It is not background. It is not even time, most people get there in six months when they're properly guided. The gap is just skill. And skill is the one thing you can actually do something about.

The industry is open. The money is there. The only question is whether you're going to break in and take it for yourself.

This book exists to make sure you choose correctly.

The Six Doors

Imagine you're standing in a hallway. At the end of the hallway there are six doors. Behind each door is a completely different career, a different set of skills, a different version of your life. All six doors lead somewhere good.

That's what choosing a tech track feels like. And for most people, standing in that hallway is overwhelming. Frontend, backend, DSA, system design, Web3, DevOps, the words alone sound intimidating before you even understand what they mean.

But here's the thing. Once you understand what's behind each door, the choice becomes a lot more obvious. Because the right door for you is the one that matches how you think, what you enjoy, and where you want to end up.

Some people love building things that look beautiful and feel great to use. Some people love solving complex logical problems that nobody else wants to touch. Some people want to be on the frontier of something brand new. Some people want to be the most essential person in any technical room they walk into.

None of these are wrong. They're just different doors.

In the next six chapters I'm going to walk you through each one. Not with jargon and technical definitions, but with stories and real pictures of what life looks like behind each door. By the end you'll know exactly which one is yours.

The hallway is behind you. The doors are in front of you. Let's go through them one by one.

The Builder's Door (Frontend Engineering)

Meet Funmilayo. She's always been the person who notices design. When she uses an app she notices when something feels off, the button is in the wrong place, the color makes her eyes hurt, the page takes too long to respond. She doesn't just use products. She experiences them.

When Temi discovered frontend engineering she said it felt like finding her language. Because frontend is exactly that, the language between a product and the person using it. Every button, every screen, every animation, every interaction. That's frontend.

Frontend engineers build what people see and touch. They work with HTML, CSS and JavaScript to create the visual layer of every website and application you've ever used. They work with frameworks like React to build interfaces that are fast, responsive and beautiful. They think about how things look and how things feel and how to make the experience so smooth that users don't even notice they're using technology.

It is the most visual track which makes it the most immediate. You build something and you can see it. You can send someone a link and they can experience what you made within seconds. That feedback loop is addictive in the best way.

The earning potential for frontend engineers ranges from ₦150,000 to ₦800,000 per month. Entry level roles include UI developer and frontend developer. As you grow you become a React developer, a product engineer, eventually a frontend lead. The roles are everywhere, every company with a product needs someone who can build it well.

The learning curve is gentler here than most other tracks. If you are a complete beginner and you want to start somewhere that will give you visible results quickly, this is often the right door.

If you want to build something and immediately see it come to life; this is your door.

The Engineer's Door (Backend Engineering)

Nobody knows David's name. That's not a bad thing. That's the point.

David is a backend engineer at a fintech company. When you open the app and send money to your friend in under three seconds, that's David. When your transaction history loads instantly and every naira is accounted for, that's David. When the app handles ten thousand transactions simultaneously without breaking a sweat, that is David's work running quietly in the background.

Backend engineering is the engine behind every product whether websites, mobile apps, or even the hardware you love. It is the logic, the data, the security, the performance. It is what makes things actually work as opposed to just looking like they work. While frontend engineers build what users see, backend engineers build what makes it possible.

Backend engineers work with programming languages like Node.js, Python and Go to build APIs, which are essentially the communication channels between different parts of a system. They design and manage databases where all the information lives. They handle authentication, which is how a system knows who you are and what you're allowed to do. They think about efficiency and security and reliability at every level.

It is more logic-heavy than frontend. The problems are less visible but more complex. And because backend work is so essential and so technically demanding, it is compensated accordingly, ₦250,000 to over ₦1,000,000 per month for backend engineers who know their craft.

Roles include backend developer, API engineer, Node developer, software engineer. Every company that has a product has a backend. Which means the demand never stops.

If you are someone who likes solving hard problems that nobody else sees and doesn't mind that the credit is quiet; this is your door.

The Thinker's Door (Data Structures & Algorithms)

Chidi failed his first technical interview. Then his second. Then his third. Three different companies, three different questions, three times walking out of the room feeling like he wasn't good enough.

He was a decent developer. He could build things. He understood the concepts. But when they put a problem in front of him and asked him to solve it efficiently, he froze. He knew how to get an answer. He didn't know how to get the right answer in the right way.

So he did something most people avoid. He spent three months doing the reps on Data Structures and Algorithms. Arrays, linked lists, trees, graphs, dynamic programming. Not because he enjoyed it; “nobody particularly enjoys it at first”, but because he understood that this was the foundation everything else was built on.

He passed his next five interviews.

DSA is not a job title. It is a weapon. It is the thing that separates developers who get filtered out in round one from developers who get offers. It is how a computer scientist thinks about problems, not just what solution works, but what solution works best, and why, and under what conditions.

When you understand DSA you start to see code differently. You understand why one approach is faster than another. You understand how to make things scale. You become the kind of developer that senior engineers want on their team because you don't just solve problems, you solve them well.

The earning potential unlocks the highest tier roles, ₦500,000 to ₦2,000,000 plus. Not because DSA alone pays that, but because DSA combined with any other track makes you a completely different calibre of developer.

Every great developer has done the reps. There are no shortcuts here. Only results.

The Architect's Door (System Design)

In 2012 a popular startup had a great product. Users loved it, pressed it, loved it. Investors loved it. Then they got featured in a major publication and three hundred thousand people tried to sign up at the same time.

The servers crashed. The app went down for six hours. People couldn't get in. Support was overwhelmed. By the time things came back online they had lost a significant chunk of the momentum they had spent months building.

The product was good. The team was talented. But nobody had thought about what happens when it actually works, when the users come and the load multiplies and the system has to handle ten times what it was built for.

That thinking, that's system design. And it's what separates engineers who can build something from engineers who can build something that lasts.

System design is how senior engineers think. It's the difference between building a product for today and building infrastructure for where the product is going. Load balancers, databases at scale, caching systems, message queues, microservices, distributed architecture, these are the tools of someone who thinks at the level of the whole system, not just their piece of it.

Most developers don't learn this until they've been working for three to five years and have made enough expensive mistakes to understand why it matters. We teach it in four weeks because there's no reason to wait for the mistakes when you can learn from other people's.

Earning potential here is ₦500,000 to ₦2,000,000 plus per month. Roles include senior developer, solutions architect, tech lead. This is the track that puts you on the path to Chief Technical Officer.

“People report to you, and you decide”

System design is where junior devs become senior devs. You're learning it now.

The Frontier Door (Web3)

Think about what it was like to build on the internet in 1995 as Tim Bernes Lee. Most people didn't understand it. Most people thought it was a passing thing. The people who built on it anyway, who saw what it could become before anyone else did, those people own the internet now.

That moment is happening again. Right now. In Web3.

Blockchain, smart contracts, decentralised finance, NFTs, DAOs; these are not just buzzwords. They are the architecture of a new layer of the internet being built in real time. One where transactions happen without banks, where ownership is provable and trustless, where code executes automatically without a middleman.

Web3 is the frontier. And like all frontiers it is chaotic, it is early, and it is full of opportunity for the people willing to show up before everyone else does.

Web3 developers build smart contracts, self-executing programs that live on the blockchain. They work with languages like Solidity for Ethereum-based chains and Rust for Solana. They build DeFi protocols, NFT platforms, on-chain games, decentralised exchanges. They are building the next version of finance, ownership, and the internet itself.

Because this space is so new and the number of people who can actually build in it properly is so small, the compensation is significant; ₦400,000 to ₦2,000,000 plus per month. Roles include smart contract developer, Web3 engineer, DeFi developer, blockchain engineer.

I build in this space personally. Solidity on Arbitrum, Rust on Solana. So when I say this is real, I mean it from experience not from research.

Web3 is still early. The people learning it now are the ones who'll be hiring everyone else in five years. The question is whether you'll be one of them?

The Operator's Door (DevOps)

It was 3am on a Friday when the alerts started coming in.

The app was down. Not slow but completely unreachable. Users were getting error pages. The support inbox was filling up. The CEO was awake and panicking. The frontend team checked their code, it was fine. The backend team checked their services, also fine. But something in the infrastructure had collapsed and nobody could figure out what.

Then Kalu joined the call. Calmly. With his coffee. He looked at the logs, traced the issue to a misconfigured load balancer, fixed it in eleven minutes, and went back to sleep.

That's a DevOps engineer. And in that moment Kalu was the most valuable person in the entire company.

DevOps is the track that nobody talks about and everybody desperately needs. DevOps engineers are responsible for the infrastructure that makes software actually run. They set up the pipelines that automatically test and deploy code. They manage the cloud environments where products live. They monitor systems, respond to incidents, and make sure that when things break, and things always break, someone who knows what they're doing is there to fix it.

They work with tools like Docker for containerisation, GitHub Actions for CI/CD pipelines, and cloud platforms like AWS and Google Cloud for deployment and scaling. They know Linux, they know bash, they know how to trace a problem through layers of infrastructure that most developers never think about.

The earning potential is ₦300,000 to ₦1,500,000 per month. Roles include DevOps engineer, cloud engineer, site reliability engineer, infrastructure engineer.

DevOps engineers don't build the product. They make sure the product never goes down. In a world where every hour of downtime costs companies real money, that is an incredibly valuable thing to be.

How to Pick Your Track

You've been through all six doors. Maybe one of them immediately felt right. Maybe you're still not sure. Either way, here's a simple framework to help you decide; three questions that will narrow it down quickly.

Question one: What do you actually enjoy doing? Not what you think you should enjoy. Not what sounds impressive. What genuinely interests you.

Do you like building things that look good? Frontend.

Do you like solving complex logical problems? Backend or DSA.

Do you like thinking about big systems and how everything fits together? System design.

Do you like being on the edge of something new? Web3.

Do you like making sure everything runs smoothly behind the scenes? DevOps.

Question two: Where do you want to end up in two years? Visualise it.

Are you a developer at a startup building products users love? That's probably frontend or backend.

Are you the most technically sharp person in any interview room you walk into? That's DSA.

Are you a senior architect designing systems that scale? System design.

Are you building the next generation of financial infrastructure like Paystack, and Paypal? Web3.

Are you the person companies call when things go wrong? DevOps.

Question three: What's your honest starting point? If you're a complete beginner who wants visible results quickly, start with frontend.

If you're already comfortable with one track and want to go deeper, DSA and system design will make everything else stronger.

If you want maximum earning potential and you're willing to do hard work; backend, Web3, and system design are your highest leverage moves.

Pick one. Start there. You can always learn more later. The worst decision you can make is no decision at all.

Your First 30 Days

Day one is not what most people expect. There is no moment where everything clicks immediately. There is no sudden feeling of being a developer. There is just you, a laptop, and a problem to solve.

And that's exactly how it's supposed to feel.

Week one is about orientation. You're learning the syntax, the environment, the tools. Things will break and you won't always know why. That's normal. The goal this week is not to understand everything. The goal is to write code every single day and get comfortable with being uncomfortable.

Week two is where it starts to connect. The syntax feels less foreign. You're spending less time looking up basic things and more time actually thinking. You build your first small project, not a tutorial project, a real one. Something you decided to build. It doesn't have to be impressive. It has to be yours.

Week three is where most people either break through or give up. The problems get harder. The solutions aren't obvious. You spend an hour on a bug that turns out to be a missing semicolon. This is the wall. The people who get through the wall are not the people who find it easy. They are the people who decide that confused and continuing is better than giving up and staying stuck.

Week four is when you start to feel it. You look at code that would have scared you three weeks ago and you understand most of it. You build something, it works, and you feel something that's hard to describe, a quiet confidence that wasn't there before.

Thirty days. That's all it takes to go from complete beginner to someone who can genuinely build things. The only variable is whether you show up every day.

The Resume Nobody Ignores

Two candidates apply for the same role. Same age, similar background, both have been learning for about six months. One gets three interview requests in a week. The other gets silence.

The difference is not their skills. It's how they presented those skills.

The resume that gets ignored looks like this, a list of technologies learned, a few online certificates, and a generic objective statement at the top that says something like "passionate developer seeking opportunities to grow."

The resume that gets interviews looks completely different. It leads with projects. Real ones, with links. It describes what each project does, what technologies were used, and what problem it solved. It has a GitHub link that actually has code in it. It is specific about what the person can do, not vague about what they're hoping to become.

The truth about tech hiring. Recruiters spend an average of six seconds on a resume before deciding whether to read further. In those six seconds they are looking for one thing, proof that you can actually do the job. Certificates and course completions are not proof. A link to something you built is proof.

So here is what your resume needs. Your name and contact information at the top, clean, no fancy formatting. Then your projects section, two or three real projects with descriptions and links. Then your skills, specific technologies, not just "programming." Then your education, briefly. That's it.

Build things. Put them on GitHub. Link them on your resume. That's the formula. Everything else is decoration.

The Door Is Open

You picked up this book because something in you already knew that tech was the move. Maybe you've been thinking about it for months. Maybe someone sent you this and it's the first time you're seriously considering it. Either way, you're here. And that matters.

Everything you've read in these pages is real. The money is real. The opportunities are real. The tracks are real. The results people like Ibraheem have gotten are real. None of this is motivation speak designed to make you feel good for ten minutes and then forget about it by morning.

This is just how the industry works. And now you know how it works too.

The only thing standing between you and the version of yourself that is cracked, employed, and earning, is the decision to start. Not the perfect time. Not when you feel ready. Not after one more thing. Now.

The cracked dev you're trying to become is already inside you. You just needed someone to show you the door.

Now you know where it is.

~ Koded from StackD